

# Farès Zaghouane

zaghouane.fares@gmail.com · github · linkedin.com/in/fares-zaghouane · Lyon, France

## SUMMARY

My work sits between representation learning and dynamical systems. At Apple I formalized hardware behaviour as reachable-state specifications; at Nokia Bell Labs I built latent video-prediction models for cloud gaming and encountered a concrete stability problem: predictions drift once the semantic anchor becomes sparse. That is the failure mode I want to study: how much structure the predictor in a JEPA-style world model requires to remain coherent over long horizons.

## RESEARCH INTERESTS

**Keywords:** World models · JEPA predictors · latent dynamical systems · state-space models · Koopman operators · representation learning · reinforcement learning, planning and control.

In joint-embedding predictive architectures the encoder decides what to represent while the predictor decides how representations evolve. The predictor is typically an unconstrained network. My question is whether replacing it with a structured latent dynamical system (SSM-style transitions, Koopman-style observables, or energy-based stability constraints), while holding the encoder and objective fixed, keeps long-horizon rollouts stable without expensive re-anchoring. The scope is narrow and the claim is falsifiable.

## EDUCATION

**École Polytechnique Fédérale de Lausanne (EPFL)** Lausanne, Switzerland

*MSc Computer Science, Machine Learning specialization — GPA 3.7/4.0* Sept 2023 – Present

*Relevant coursework:* Artificial Neural Networks & Reinforcement Learning, Computer Vision, Machine Learning, Natural Language Processing, ML in Finance, Cryptography & Security.

**CPE Lyon** Lyon, France

*Diplôme d'Ingénieur (MSc-equivalent), Computer Science — Software Engineering & Big Data — GPA 3.6/4.0* Sept 2021 – Present

*Relevant coursework:* Software architecture, Design patterns, Databases, Data mining, Embedded systems, System administration, Quantum computing.

**CPE Lyon** Lyon, France

*Licence (BSc), Computer Science & Electronics — GPA 3.6/4.0* Sept 2020 – Jul 2021

*Relevant coursework:* Linear algebra, Algorithms, Signal processing, Digital & analog electronics, Discrete & continuous mathematics, Quantum mechanics.

## EXPERIENCE

**Nokia Bell Labs** Stuttgart, Germany

*Research Intern (MSc Thesis)* Aug 2025 – Feb 2026

- The goal was to predict dense focus maps from cloud-gaming video for bandwidth and quality allocation under strict per-frame latency constraints, using a semantic signal as teacher and a lightweight model for deployment.
- Built a teacher–student video-prediction pipeline combining Grounding DINO semantic anchoring, RAFT optical-flow propagation, and a lightweight correction network to produce dense focus maps.
- Ran experiments with JEPA-style latent prediction, state-space models, and Mamba-Vision encoders; settled on a hybrid that met the real-time latency budget while matching semantic-only accuracy closely.
- Designed the pseudo-supervision setup, tracked temporal drift across pipeline modes, and built the dense-map evaluation protocol; the hybrid recovered most of the semantic-only quality at a fraction of the cost and outperformed motion-only and saliency-only baselines.

**Apple** Munich, Germany

*Design Verification Research Intern* Oct 2022 – Jul 2023

- Modeled IP behavior using finite-state machines and event-based abstractions, allowing long-running processes

to be reasoned about without expanding every low-level timing step.

- Applied model checking to behaviors that simulation cannot cover exhaustively.
- Built Python tooling that generates Verilog/SystemVerilog assertions from high-level specs and validated it on industrial designs; FSM-level specs feed directly into the model-checker.
- The event-based abstraction set the level at which reachable states are specified: model checking operates there rather than at the cycle level, which changed which properties were practical to verify.

## PROJECTS

---

### **GPT-2 from Scratch**

PyTorch implementation of a GPT-2-style decoder-only Transformer, built against the original papers: tokenization, token and positional embeddings, causal and multi-head attention, transformer blocks, pretraining, generation, and fine-tuning.

### **Stable Diffusion from Scratch**

PyTorch reconstruction of latent diffusion built against the original papers: VAE encoder/decoder, CLIP-style text conditioning, U-Net denoiser, DDPM sampler, and generation pipeline.

### **Vision Transformer from Scratch**

From-scratch ViT covering image patching, self-attention, representation learning, and classification; extends the GPT-2 transformer architecture to images.

### **Machine Learning from Scratch**

Classical ML and deep learning rebuilt from first principles — regression, k-NN, MLPs, optimizers — with coverage extending toward CNNs, transformers, SSMs, and RL; each topic starts as a notebook and is refactored into reusable modules.

### **Cart-Pole Q-Learning**

Full Q-learning agent for Cart-Pole: environment handling, training entry point, configuration files, model storage, figures, and tests.

### **V-JEPA2 ROI Semantic Extraction**

Research prototype that organizes V-JEPA-style predictive video representations around semantic regions rather than uniform spatial patches.

### **VLM Data Engine**

Video data pipeline covering shard processing, FPS conversion, scene detection, clip extraction, annotation, and quality checks; built to support multimodal world-model experiments.

## SKILLS

---

**ML & representation learning:** PyTorch, TensorFlow, Neural networks, Self-supervised learning, JEPA / world models, Energy-based models

**Vision & multimodal systems:** Computer vision, Video understanding, Grounding DINO, RAFT / optical flow, Focus-map prediction, Multimodal data pipelines

**Dynamics, control & RL:** Reinforcement learning, State-space models, Mamba, Koopman operators, Dynamical systems, Physics-informed AI

**Mathematical & scientific foundations:** Linear algebra, Applied statistics, Signal processing, Optimization, Spectral methods, Functional analysis

**Software & formal methods:** Python, NumPy, Jupyter, Formal verification, Model checking, Finite-state machines, Verilog / SystemVerilog assertions